

Research topics

Konkuk University, Seoul

Christian Rinderknecht

18 October 2004

Illustrated professional overview

Christian Rinderknecht

In short

- ▶ Ph.D. at INRIA (Rocquencourt) in the Cristal team (<http://cristal.inria.fr>) from 1993 to 1997.
Application of formal techniques to ASN.1, a specification language used in telecommunications (design and development of a compiler front-end and soundness proof of an encoding rule). **Theoretical computer science background.**
- ▶ Alternating academic and industrial positions, with common point the modelisation of technical problems, conception of solutions and software development. From Sep. 2001 to Sep. 2002, researcher at *Information and Communications University* (Daejeon).
- ▶ From 2003 until Sep. 2004, assistant professor at a French private engineering school (<http://www.devinci.fr>).

Research topics

- ▶ Protocol engineering in telecoms (INT, ICU, de Vinci): specification analysis, formal design review, methods and software tools for automated test suite generation. ($\approx$ four years)
- ▶ Software development cycle (Alcatel, PolySpace): compilation, static program analysis, tests, code quality ($\approx$ two years).
- ▶ At *Léonard de Vinci*, XML-based web-site frameworks supporting e-learning, but not exclusively. ($\approx$ one year)
- ▶ In all cases, my work was to lead and apply some research, the approach was rigourous (sometimes formal), often guided by the **“language” aspect of the problems**.

Projects and collaborations

- ▶ During my Ph.D. I collaborated closely with France Telecom R&D. Since then I have been in contact with the ASN.1 Project leader at the International Telecommunication Union (ITU).
- ▶ At the French National Institute of Telecommunications (INT) I took part to two national projects, involving both academic and industrial partners.
- ▶ At *Léonard de Vinci*, I was a member of the European project Kaleidoscope (e-learning, XML frameworks, semantic web, ontologies etc.) and of the AIDA seminar (*Interdisciplinary Approach for Interactive Learning Environments*).

Teaching

- ▶ During my Ph.D. (1993-1996), I taught at univ. René Diderot (Paris VII) to undergraduates: algorithmics, Pascal, C++, linear algebra, integral calculus. I designed and tutored student projects, prepared exams etc.
- ▶ Teaching assistant at *Institut Supérieur d'Électronique de Paris*, an engineering school: C++ programming (1993-1996).
- ▶ Teaching assistant in algorithmics and functional programming with Caml (1996-1997).

Teaching (continued)

- ▶ Professional courses: Pascal (1993) and C++ (1999).
- ▶ At INT, an engineering school: supervision of last-year projects and tutoring of a master student; teaching of *Semantics Definition Language* (SDL) to students and professionals (in Spanish), from 1998 to 2000.
- ▶ At *Léonard de Vinci*, I taught software engineering, introduction to formal methods, Java programming, Objective Caml (functional programming), Unix shell, gnu development tools, compiler generation, introduction to protocol engineering. I supervised student's projects and training periods in companies.

Visiting Professor at Information and Communications University (2001-2002)

- ▶ Full-time research position. I published a journal paper about ASN.1. I learnt new issues in mobile networks, like *ad hoc* networking.
- ▶ *Abstract Syntax Notation One* (ASN.1) is a standard language for definition of types whose values may be exchanged at run-time by communicating applications, independently of the possible heterogeneity of the peers. Applications: network equipment management (SNMP), e-mail (X.400), mobile telephony, real-time multimedia communication over the internet (MHEG, H.225, H.245, H.323), fax over the internet (T.38), cryptographic protocols (SET, OpenSSL, PKCS 7, Kerberos, biometrics) and databases (X.500 directory, Z39.50 (U.S. Library of Congress), DNA repositories).
- ▶ I solved formally the important problem of validating ASN.1 specifications by borrowing technical concepts from another field (set constraints).

Assistant Professor at *Léonard de Vinci* (2003-2004)

- ▶ I published two papers on two very different topics:
 - ▶ a question-answering mining web-platform,
 - ▶ the formal design review of ASN.1 and BER.
- ▶ ASN.1 is a high-level data (*versus* control) notation for protocols
- ▶ consisting of data and type definitions
- ▶ and network concepts modeling features (like PDUs),
- ▶ embedded in other specification languages like SDL (control), MSC (functional requirements), TTCN (test suites) etc.
- ▶ also used without formal control specification by protocol designers and tool implementors, e.g. SNMP, OpenSSL.

What are the Basic Encoding Rules (BER)?

- ▶ The BER are a set of rules defining the serialization into bits of ASN.1 values;
- ▶ the bit frames (*codes* or *encodings*) have a uniform structure: a triple made of a *tag*, a *length* and some *contents*;
- ▶ the tag field is extracted from the data type,
- ▶ the contents field is extracted from the data itself,
- ▶ the length field is computed over the contents field,
- ▶ the structure allows embedding of sub-frames (in the contents field)
- ▶ and an *indefinite length* form, as a sender's option.

How do ASN.1 and the BER interact?

- ▶ Both peers share a common ASN.1 specification and agree upon the usage of the BER;
- ▶ each peer compile the ASN.1 specification to a set of type declarations and a set of coders/decoders (codecs) for each type;
- ▶ the target language may be different depending on the peer;
- ▶ the coders transform a value into a stream of bits according to the BER,
- ▶ the decoders transform a stream of bits into a value according to the BER (or reject it as ill-formed);
- ▶ these pieces of source code are compiled and linked separately against the corresponding communicating applications.

So, what is wrong?

- ▶ In the last few years, the press has reported several alleged vulnerabilities
- ▶ relating ASN.1 and the BER to SNMP and OpenSSL
- ▶ about improper decoding (and rejection) of ill-formed BER encodings
- ▶ leading to buffer overflows,
- ▶ unspecified (non-deterministic) behaviours,
- ▶ stack corruptions,
- ▶ hence possible denials of service.
- ▶ Further study proved that all the problems lay actually in *implementations*:
- ▶ **we want to show that ASN.1 and the BER are, themselves, well designed.**

Modeling (assumptions and encoding)

We start making some assumptions that abstract away low-level details and help us capturing the design principles of the BER:

- ▶ values are given in ASN.1, not in the application language (in fact they are produced in memory at run-time);
- ▶ codes are tuples, not bit series, in order to easily reason by induction on their structure;
- ▶ the network is reliable (no alteration of codes).

BER codes may not be unique for a given value, due to possible sender's options (e.g. encoding of booleans or unordered sets).

Proposition

All the BER encodings of a given value, according to a given type, are equivalent.

Modeling (decoding)

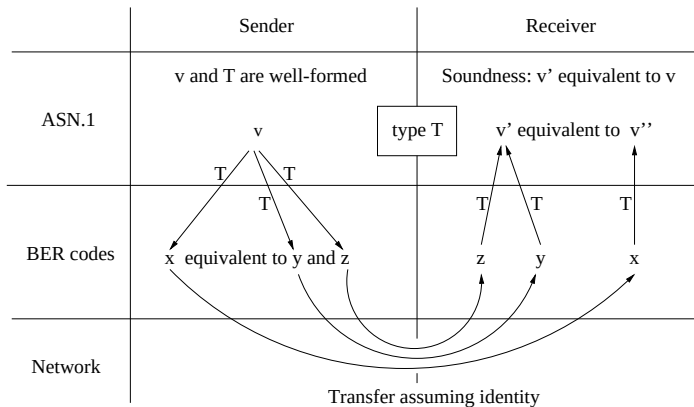
- ▶ The BER standard (26 pages of English) does not define the decoding at all.
- ▶ We believe that the receiver is the peer most concerned with the sound design of the BER: **the composition of encoding and decoding yields a value equivalent to the original one.**

Theorem (Soundness)

Let v be a value of type T . Then the BER decoding of any BER encoding of v is equivalent to v .

Hence we need an equivalence relationship between values (thus independent from the BER).

Soundness property



Equivalence entailment

As a consequence, in order to prove the soundness property, it appears that we need to link logically the two equivalence relationships we defined separately on codes and values.

Proposition (Equivalence entailment)

Let c_1 and c_2 be two equivalent codes. Then the decoding of c_1 is equivalent to the decoding of c_2 assuming the same type.

One equivalence rely solely on the BER and the other one is inherent to ASN.1. Hence **this proposition is the link between the design of ASN.1 and the design of the BER.**

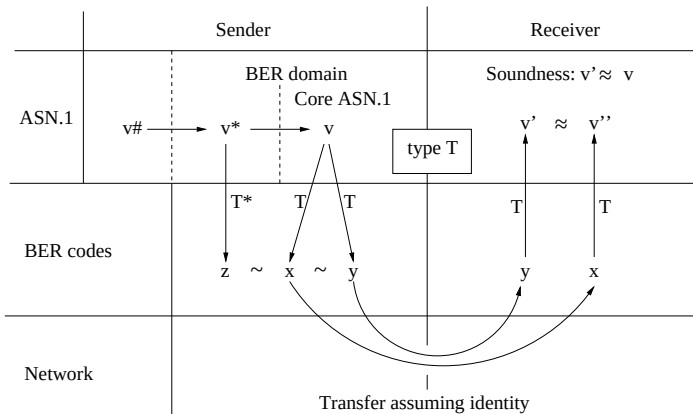
Core ASN.1

- ▶ The BER only apply to a subset ASN.1 (whose definition spans 188 pages).
- ▶ This suggests that the whole ASN.1 can be reduced to an inner subset which has the same expressivity, we call **BER domain**.
- ▶ In fact it is even possible to reduce further the BER domain to a smaller subset, we call **core ASN.1**, in which formal reasoning is easier.
- ▶ The formal definition of these reductions is achieved by means of a series of rewriting rules that preserve the set of values of the specifications (or of a given type).
- ▶ We can easily check some useful properties in core ASN.1 and enforce e.g.

Theorem (BER termination)

The encoding of core ASN.1 values with the BER always terminates.

Core ASN.1 and soundness property



Getting formal

We define the encoding with a *system of inference rules*. These are logical implications $P_1 \wedge P_2 \wedge \dots \wedge P_n \Rightarrow C$ graphically represented as

$$\frac{P_1 \quad P_2 \quad \dots \quad P_n}{C}$$

where the P_i are the *premises* and C is the *conclusion*. When there is no premise, C is an *axiom* and is simply noted C .

An inference rule can be interpreted also from a computational point of view: in order to compute C , we need to compute the P_i first (order is left unspecified).

Inference rules (continued)

The rules and axioms can contain unquantified variables (*free variables*). In this case they are implicitly universally quantified ($\forall$) at the beginning. For instance

$$\frac{P_1(x) \quad P_2(y)}{P(x, y)} \text{ Prop}$$

actually denotes the property Prop which is $\forall x, y. P_1(x) \wedge P_2(y) \Rightarrow P(x, y)$.

A system of inference rules is a non-ordered set of inference rules. Given a system applying to one or more relations, these are defined as the smallest relations satisfying the rules.

Smallest relations

Let the following rules applying to the predicates $\text{Even}(n)$ and $\text{Odd}(n)$:

$$\text{Even}(0) \qquad \frac{\text{Odd}(n)}{\text{Even}(n+1)} \qquad \frac{\text{Even}(n)}{\text{Odd}(n+1)}$$

By definition, we understand them as the propositions

$$\begin{aligned} &\text{Even}(0) \\ &\forall n. \text{Odd}(n) \Rightarrow \text{Even}(n+1) \\ &\forall n. \text{Even}(n) \Rightarrow \text{Odd}(n+1) \end{aligned}$$

Many predicates satisfy these conditions, e.g. $\text{Even}(n)$ and $\text{Odd}(n)$ true for all n . But the smallest satisfactory predicates (the ones true the less often) are $\text{Even}(n) \triangleq (n \bmod 2 = 0)$ and $\text{Odd}(n) \triangleq (n \bmod 2 = 1)$.

Proof trees

A theorem is a judgement, i.e. a formal statement, a formula. A proof of a theorem is a tree whose root (the conclusion) is the theorem, the inner nodes are the conclusions of its subtrees and the leaves are axioms. Such a tree is called a *proof tree*. The conclusion is drawn at the bottom of the page. For instance, here is the proof of $\text{Odd}(3)$ with the previous system:

$$\frac{\frac{\frac{\text{Even}(0)}{\text{Odd}(1)}}{\text{Even}(2)}}{\text{Odd}(3)}$$

Back to our model

- ▶ Let us note $\Gamma \vdash v : T \rightarrow c$ the judgement “In the environment Γ , the value v is encoded into the code c , following the type T .” (The environment models the ASN.1 module.)
- ▶ Let us note $\mathcal{D}(\Gamma, c, T)$ the decoding of c in the environment Γ according to type T .
- ▶ We already noted $v_1 \approx v_2$ the equivalence of values v_1 and v_2
- ▶ and $c_1 \sim c_2$ the equivalence of codes c_1 and c_2 .

The next step consists in defining these relations by means of systems of inference rules.

Formal model

We can now rephrase formally our propositions:

- ▶ Proposition “All the BER encodings of a given value, according to a given type, are equivalent.” becomes

Proposition

If $\Gamma \vdash v : T \rightarrow c_1$ and $\Gamma \vdash v : T \rightarrow c_2$ then $c_1 \sim c_2$.

- ▶ Proposition “The decoding of two equivalent codes lead to two equivalent values.” becomes

Proposition (Equivalence entailment)

$c_1 \sim c_2 \implies \mathcal{D}(\Gamma, c_1, T) \approx \mathcal{D}(\Gamma, c_2, T)$

- ▶ Theorem “Let v be a value of type T . Then the BER decoding of *any* BER encoding of v is equivalent to v .” becomes

Theorem (Soundness)

If $\Gamma \vdash v : T \rightarrow c$ then $\mathcal{D}(\Gamma, c, T) \approx v$.

Formal methods for the customer

- ▶ We prove the soundness theorem with a technique called **structural induction** (on the height of the proof trees) and **case analysis**.
- ▶ In case we are a standardization organisation or a tool seller we can say to the BER users: “The formal design review concludes you can be pretty confident on the technology you use.”
- ▶ If we want a higher level of confidence, we can go further by refining our model, i.e. stating explicitly what was abstracted away and weakening assumptions.

Formal methods for the software company

- ▶ In case we are a tool implementor, the *reliable* back-end of an ASN.1 compiler can be derived from the previous work. Indeed our model, being based on inference rules, is actually close to an **operational model**, i.e. a real implementation, **based on a functional language**.
- ▶ In this case we do not need to publicise to our customer the usage of formal methods: these will ensure that the kernel of our software conforms to a formal specification, hence will have less bugs and will be easier to maintain, in the end.
- ▶ In industries other than the software industry, such formal design reviews are part of the normal process, e.g. the study of electromagnetic fields interference for the new Incheon subway line.